Descriptive Statistics — Exploring a PostgreSQL Database on Azure

From database connection to plots, hypothesis tests, and regression

Teaching Document

2026-07-14

Table of contents

0. Setup — install the packages . . . 2
1. Connect to Azure PostgreSQL . . . 3
    1.1 What tables are available? . . . 4
2. Load the data into pandas . . . 5
    2.1 Derived columns . . . 6
3. Describing data with numbers . . . 7
    3.1 A caution: free-text categories are messy . . . 8
4. Plot styling . . . 9
5. The distribution of one numeric variable . . . 10
    5.1 Histogram . . . 10
    5.2 Density (KDE) plot . . . 11
    5.3 ECDF plot . . . 12
6. Comparing categories . . . 13
    6.1 Bar chart — ordered categories . . . 13
    6.2 Bar chart — unordered categories, sorted . . . 14
    6.3 Box plot . . . 15
    6.4 Violin plot . . . 16
7. Relationships and change over time . . . 17
    7.1 Scatter plot . . . 17
    7.2 Line / step plot over time . . . 18
    7.3 Correlation heatmap . . . 19
    7.4 Pair plot . . . 20
8. Hypothesis testing . . . 21
    8.0 Prepare the features . . . 22
    8.1 Two-sample t-test — do two group means differ? . . . 23
    8.2 Chi-square test — are two categorical variables associated? . . . 24
9. Regression . . . 25
    9.1 Simple linear regression . . . 25
    9.2 Multiple regression — adding a categorical predictor . . . 26
    9.3 Logistic regression — when the outcome is yes/no . . . 29
    9.4 Exercises . . . 32

10. Cheat sheet 32
Which plot for which question? 32
Which test / model for which question? 32

> **Note**
>
> Rendering this document runs every query live. Before `quarto render`, set the database password once in your terminal so the connect cell finds it: `$env:PGPASSWORD = "<password>"` (PowerShell) or `export PGPASSWORD=...` (bash).

Welcome! This notebook is a hands-on tour of basic data analysis, using **real data from our class**. Step by step, you will learn how to:

1. **Connect to a PostgreSQL database hosted on Azure** — the same database our project-matching website uses,
2. **Pull data into Python** with SQL queries,
3. **Describe the data with numbers and plots** — histograms, bar charts, box plots, scatter plots, and more — and learn *which question each plot answers*,
4. **Test ideas with statistics** — hypothesis tests, and linear & logistic regression.

You do not need any prior statistics knowledge. Every new term is explained the first time it appears.

The data. The `classproject` database is the storage behind our class **project-matching platform** (the website where students post projects and apply to join them). It contains these tables:

Table	Rows (July 2026)	What one row means
`accounts_user`	59	one registered student: department, major, languages, gender, birth year, study year, height
`projects_project`	18	one proposed project: title, how many members it wants, status
`projects_application`	34	one application: "student X asked to join project Y"
`projects_membership`	30	one confirmed membership: "student X is in project Y"

(There are also `reports_report` and `reports_peerreview` tables, but they are still nearly empty — nothing to analyze yet.)

A note on privacy. This is *real classmate data*. We deliberately never load the `password`, `email`, or `full_name` columns, and every figure below shows groups, not individuals — no plot can identify a single person.

0. Setup — install the packages

Python itself cannot talk to databases or draw plots; we borrow that ability from **packages** (add-on libraries). The cell below installs them — you only need to run it once per computer, and it does nothing if they are already installed:

- `sqlalchemy` + `psycopg2-binary` — connect to and talk to PostgreSQL
- `pandas` — hold data in tables and run SQL queries
- `matplotlib` + `seaborn` — draw plots
- `scipy` + `statsmodels` — hypothesis tests and regression

```
# One-time setup -- run in a notebook or terminal if packages are missing
%pip install --quiet pandas sqlalchemy psycopg2-binary matplotlib seaborn scipy statsmodels
```

1. Connect to Azure PostgreSQL

Our database does not live on your laptop — it runs on a Microsoft Azure server on the internet. To use it from Python we need the same five pieces of information that any database client (including the VS Code PostgreSQL extension) needs. Think of it as a postal address plus a key:

Setting	Value	Analogy
Host	`classproject-iss-bfsu-db.postgres.database.azure.com`	the building's street address
Port	`5432`	which door to knock on (5432 is PostgreSQL's standard door)
Database	`classproject`	which apartment inside the building
User	`classadmin`	whose name is on the key
Password	*(not written here!)*	the key itself

You can see these values in VS Code: PostgreSQL sidebar → right-click the connection → *Edit Connection*.

Two habits that matter in practice:

- **Never type the password into the notebook.** This file is saved in a git repository — anything written here can be seen by everyone with access to the repo, forever. Instead, the code below reads the password from an *environment variable* called `PGPASSWORD` if it exists, and otherwise asks you to type it into a hidden prompt (`getpass`). Either way, the password never appears in the file.
- **Passwords must be URL-encoded.** The connection is written as one long address (a URL). If your password contains characters like `&`, `@` or `:`, they would break the URL — `quote_plus()` safely escapes them.

One more term you will see: `sslmode=require` tells the connection to be **encrypted** (SSL), so nobody between your laptop and Azure can read the traffic. Azure refuses unencrypted connections.

```
import os
from getpass import getpass
from urllib.parse import quote_plus

from sqlalchemy import create_engine, text
import pandas as pd
```

```
HOST = "classproject-iss-bfsu-db.postgres.database.azure.com"
PORT = 5432
DBNAME = "classproject"
USER = "classadmin"

# Prefer the PGPASSWORD environment variable; otherwise ask (input stays hidden).
password = os.environ.get("PGPASSWORD") or getpass(f"Password for {USER}: ")

url = (
    f"postgresql+psycopg2://{USER}:{quote_plus(password)}@{HOST}:{PORT}/{DBNAME}"
    "?sslmode=require"          # Azure requires encrypted connections
)

# The "engine" is our reusable door to the database.
# pool_pre_ping re-checks idle connections -- Azure closes them after a while.
engine = create_engine(url, pool_pre_ping=True)

with engine.connect() as conn:
    print("Connected to:", conn.execute(text("SELECT version();")).scalar())
```

```
Connected to: PostgreSQL 16.14 on x86_64-pc-linux-gnu, compiled by gcc (GCC) 13.2.0, 64-bit
```

If the cell prints a `PostgreSQL 16.x ...` version string, you are connected. If not, the error message usually tells you exactly what is wrong:

- **`timeout expired` / `could not connect`** — Azure's firewall does not know your current internet address (IP). Every new network (home vs. campus vs. phone hotspot) has a different IP, and the server only accepts IPs on its allow-list. Add yours with the Azure CLI (or Azure Portal → the server → *Networking*):

  ```
  az postgres flexible-server firewall-rule create \
      --resource-group classproject-rg --server-name classproject-iss-bfsu-db \
      --name allow-my-ip --start-ip-address <your.public.ip>
  ```

- **`password authentication failed`** — the password is wrong (watch for spaces before/after when pasting).
- **`no pg_hba.conf entry ... no encryption`** — the `?sslmode=require` part is missing from the URL.

1.1 What tables are available?

A database can describe itself: the built-in `information_schema.tables` view is a directory of every table. This is always a good first query on an unfamiliar database.

```
pd.read_sql(
    """
    SELECT table_name
    FROM information_schema.tables
```

```
    WHERE table_schema = 'public' AND table_type = 'BASE TABLE'
    ORDER BY table_name;
    """,
    engine,
)
```

	table_name
0	accounts_user
1	accounts_user_groups
2	accounts_user_user_permissions
3	auth_group
4	auth_group_permissions
5	auth_permission
6	django_admin_log
7	django_content_type
8	django_migrations
9	django_session
10	projects_application
11	projects_membership
12	projects_project
13	reports_peerreview
14	reports_report

The `django_*`, `auth_*` and `accounts_user_*` tables are internal plumbing of the web framework — our data lives in `accounts_user`, `projects_*`, and `reports_*`.

2. Load the data into pandas

`pd.read_sql(query, engine)` sends any SQL query to the database and returns the result as a **DataFrame** — pandas' table type, with rows and named columns, that all our plotting functions understand.

Note what the queries below do *not* select: sensitive columns (`password`, `email`, `full_name`) are never loaded, and the one staff/admin account is filtered out so the user table contains only students.

```
users = pd.read_sql(
    """
    SELECT id, department, major, languages, gender,
           birth_year, study_year, height_cm, date_joined
    FROM accounts_user
    WHERE is_staff = FALSE;      -- students only, drop the admin account
    """,
    engine,
)

projects = pd.read_sql(
    "SELECT id, title, group_size, status, created_at, owner_id FROM projects_project;",
```

```
    engine,
)

applications = pd.read_sql(
    """
    SELECT id, status, discussed_with_owner, created_at, decided_at,
           applicant_id, project_id
    FROM projects_application;
    """,
    engine,
)

memberships = pd.read_sql(
    "SELECT id, joined_at, member_id, project_id FROM projects_membership;",
    engine,
)

print(f"users {users.shape}, projects {projects.shape}, "
      f"applications {applications.shape}, memberships {memberships.shape}")
users.head(3)
```

```
users (58, 9), projects (18, 6), applications (34, 7), memberships (30, 4)
```

	id	department	major	languages	gender	birth_year	study_year	height_cm	date_joined
0	17	English	English	Chinese, English	F	2006	2	163	2026-07-07
1	3	English	non-fiction	English, Spanish	M	2002	3	168	2026-07-05
2	13	English	English	Chinese, English	M	2005	4	180	2026-07-07

`.shape` is `(rows, columns)`, and `.head(3)` shows the first three rows — always peek at your data after loading it.

2.1 Derived columns

Raw columns are rarely exactly what we want to analyze, so we compute a few new ones (this is often called *feature engineering*):

- **age** from birth year,
- a readable **gender label** (the raw column has `'F'`, `'M'`, one blank, and one `'N'` — we group the rare values into "Other / unspecified" because a category of size 1 cannot be summarized),
- for each project, **how many applications it received** and **how many members it has**.

One idea to keep in mind throughout: the **grain** of a table = what one row stands for. `users` has one row *per student*; `projects` one row *per project*. Every plot below is built on one of these grains, and mixing them up is the most common beginner mistake.

```
THIS_YEAR = pd.Timestamp.now().year
users["age"] = THIS_YEAR - users["birth_year"]

users["gender_label"] = (users["gender"].map({"F": "Female", "M": "Male"})
                                        .fillna("Other / unspecified"))

# For each project: count its rows in the applications / memberships tables
projects["n_applications"] = (projects["id"]
                              .map(applications["project_id"].value_counts())
                              .fillna(0).astype(int))
projects["n_members"] = (projects["id"]
                         .map(memberships["project_id"].value_counts())
                         .fillna(0).astype(int))
projects["fill_rate"] = (projects["n_members"] / projects["group_size"]).round(2)

projects[["title", "group_size", "n_applications", "n_members", "fill_rate", "status"]].head()
```

	title	group_size	n_applications	n_members	fill_rate	status
0	Research Report on the Impact of AI on [a Spec...	2	1	1	0.50	fulfille
1	LifeHub	3	2	2	0.67	fulfille
2	InfoFlow Simulator: A Telemetry Dashboard for ...	4	4	3	0.75	fulfille
3	Official Discourse Construction on AI Governan...	4	4	3	0.75	fulfille
4	Car Culture and Student Interest Data Tracker	2	0	0	0.00	open

3. Describing data with numbers

Before drawing anything, look at summary numbers. For **numeric** columns, `describe()` reports:

- **count** — how many non-missing values there are,
- **mean** — the average,
- **std** — the *standard deviation*: roughly, how far a typical value sits from the average (small std = values huddle together, large std = values spread out),
- **min / max** — the extremes,
- **25% / 50% / 75%** — the *quartiles*: the values that cut the sorted data into quarters. The 50% value is the **median** — the middle value, with half the students below and half above.

```
users[["age", "study_year", "height_cm"]].describe().round(2)
```

	age	study_year	height_cm
count	58.00	58.00	58.00
mean	20.29	2.02	170.60
std	1.38	0.87	6.94
min	18.00	1.00	158.00
25%	19.00	1.00	165.00
50%	20.00	2.00	170.00

	age	study_year	height_cm
75%	21.00	2.00	175.00
max	26.00	4.00	185.00

For **categorical** columns (labels rather than numbers), quartiles make no sense — instead we count how often each label occurs with `value_counts()`:

```
users["gender_label"].value_counts()
```

```
gender_label
Female                 35
Male                   22
Other / unspecified     1
Name: count, dtype: int64
```

And we can combine both ideas: split the students into groups, then summarize a number *within each group*. This table is exactly what the box plot in section 6 will draw:

```
(users.groupby("gender_label")["height_cm"]
      .agg(["count", "mean", "median", "std", "min", "max"])
      .round(1))
```

gender_label	count	mean	median	std	min	max
Female	35	166.9	167.0	4.9	158	178
Male	22	177.0	178.0	4.6	168	185
Other / unspecified	1	160.0	160.0	NaN	160	160

3.1 A caution: free-text categories are messy

`department` and `major` were typed by hand during registration. The same department therefore appears under many different spellings and languages — for example 国际新闻与传播学院, `Journalism`, `journalist`, and `School of International Journalism and Communication` are arguably all one group, but the computer sees four different labels. Look:

```
users["department"].value_counts().head(10)
```

```
department
国际商学院                                                  6
国际新闻与传播学院                                                 6
English                                                 5
北京外国语大学                                                   4
6528                                                    2
```

```
School of Information Science and Technology    2
1                                               2
French                                          2
German                                          2
law                                             2
Name: count, dtype: int64
```

The lesson: never group by a free-text column without cleaning it first — the counts are technically correct but meaningless as categories. Cleaning this properly means building a mapping table (every raw spelling → one canonical name), which is a worthwhile exercise but not our topic today. We will stick to the *structured* columns (`gender`, `study_year`, `birth_year`, `height_cm`) for grouped analysis.

4. Plot styling

One setup cell so every figure looks consistent and is easy to read. Two choices worth understanding:

- **Colors are fixed per category.** "Female" is the same blue in every single figure, "Male" the same green. If colors changed from plot to plot, your reader would have to re-learn the legend every time.
- The palette is **colorblind-safe** (distinguishable for people with the most common forms of color-vision deficiency, roughly 1 in 12 men).

Everything else is cosmetics: light gridlines, no box around the plot, a font that can also display Chinese characters (some project titles contain them).

```
import matplotlib.pyplot as plt
from matplotlib.ticker import MaxNLocator
import seaborn as sns

# Ink & chrome
INK, INK2, MUTED = "#0b0b0b", "#52514e", "#898781"
GRID, SURFACE, BASELINE = "#e1e0d9", "#fcfcfb", "#c3c2b7"

# One accent color for single-series plots
BLUE = "#2a78d6"

# Fixed colors for the gender groups, reused in every figure below
GENDER_ORDER = ["Female", "Male", "Other / unspecified"]
GENDER_COLORS = {"Female": "#2a78d6", "Male": "#1baf7a",
                 "Other / unspecified": "#898781"}

sns.set_theme(style="ticks", rc={
    "figure.facecolor": SURFACE, "axes.facecolor": SURFACE,
    "axes.edgecolor": BASELINE, "axes.labelcolor": INK2, "text.color": INK,
    "xtick.color": MUTED, "ytick.color": MUTED,
    "axes.grid": True, "grid.color": GRID, "grid.linewidth": 0.8,
    "axes.spines.top": False, "axes.spines.right": False,
    "figure.dpi": 110, "figure.figsize": (7.5, 4.2),
```

```
    # fall back to a Chinese-capable font for CJK characters in labels
    "font.sans-serif": ["DejaVu Sans", "Microsoft YaHei", "sans-serif"],
})
```

5. The distribution of one numeric variable

"Distribution" simply means: *what values does a variable take, and how often?* These three plots answer that question in different ways.

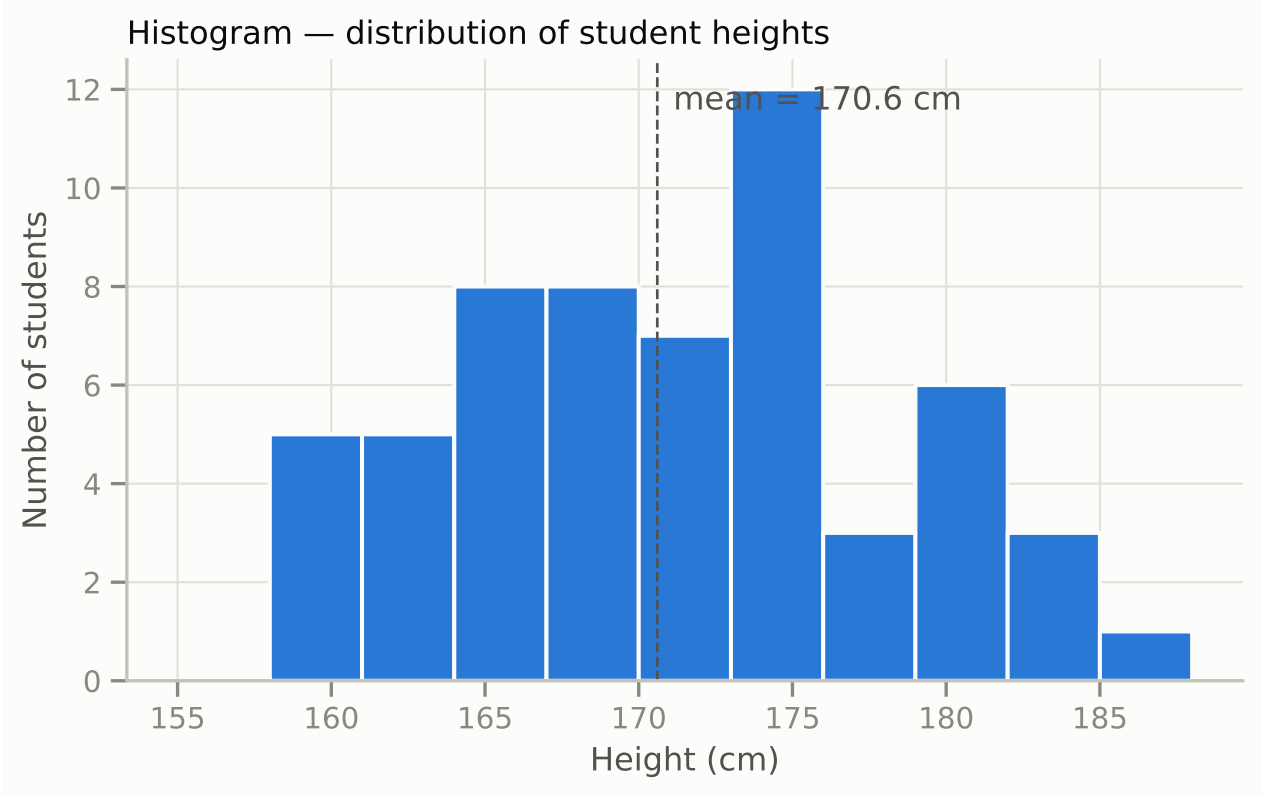
5.1 Histogram

The histogram chops the number line into equal intervals (**bins**) and counts how many students fall into each. Tall bar = common values.

How to read it: the x-axis is the variable (height), the y-axis is a count of students. Look for: where is the bulk? is it symmetric or lopsided (*skewed*)? are there two humps (two subgroups hiding inside)? any lonely bars far from the rest (*outliers*)?

One warning: the picture depends on the **bin width** you choose. Too few bins hides structure; too many shows random noise. Try changing `3` below to `1` or `10` and see.

```
fig, ax = plt.subplots()
ax.hist(users["height_cm"].dropna(), bins=range(155, 190, 3),
        color=BLUE, edgecolor=SURFACE, linewidth=1.5)

mean_h = users["height_cm"].mean()
ax.axvline(mean_h, color=INK2, linestyle="--", linewidth=1)
ax.annotate(f"mean = {mean_h:.1f} cm", xy=(mean_h, ax.get_ylim()[1] * 0.92),
            xytext=(6, 0), textcoords="offset points", color=INK2)

ax.set_xlabel("Height (cm)")
ax.set_ylabel("Number of students")
ax.set_title("Histogram —— distribution of student heights", loc="left")
plt.show()
```



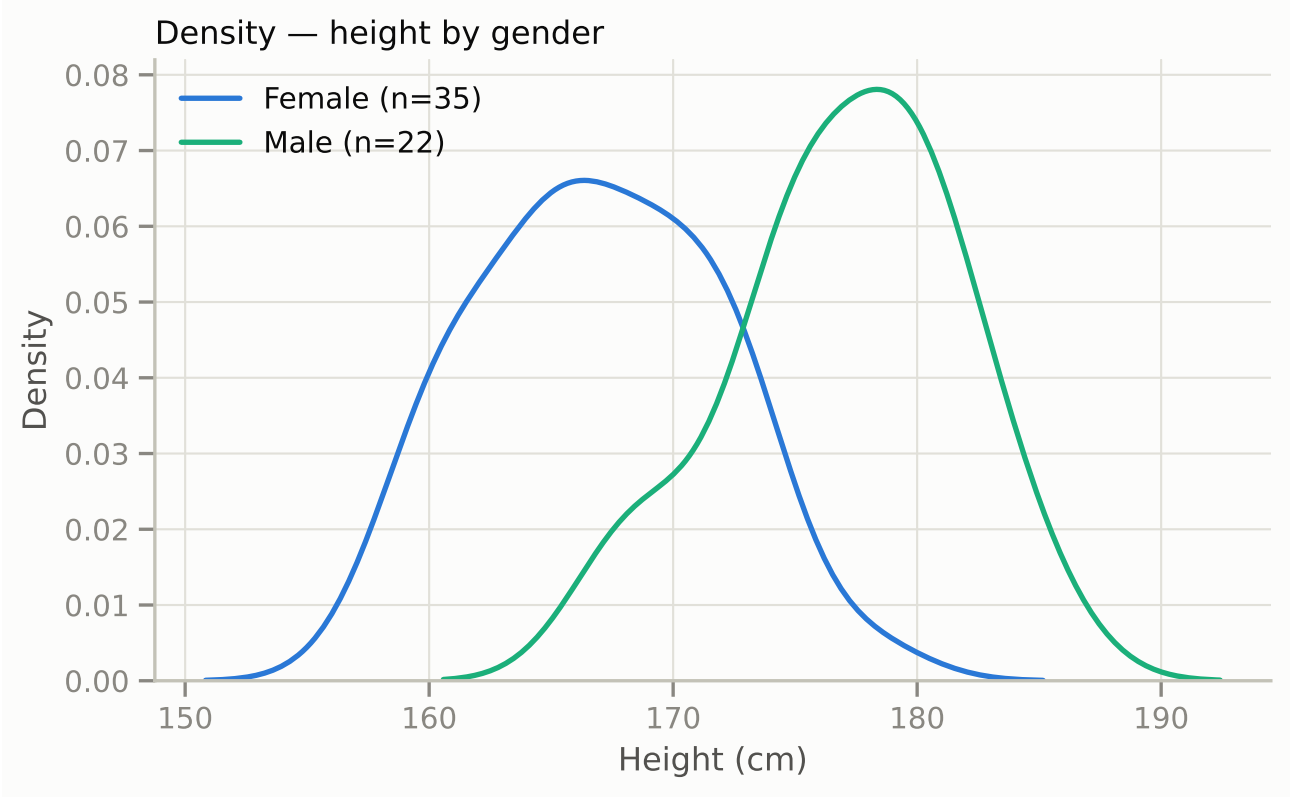
5.2 Density (KDE) plot

A **kernel density estimate** is a smoothed-out histogram: instead of bars, a smooth curve traces where values are common (high curve) and rare (low curve). The y-axis is "density", scaled so the area under the curve is 1 — read the *shape*, not the exact y-values.

Its real strength is **comparing groups**: two smooth curves overlay cleanly where two sets of bars would be a mess. Here we compare heights of female and male students. Notice the curves overlap a lot — plenty of students of every height in both groups — but their peaks sit about 10 cm apart. Keep this picture in mind: sections 8 and 9 will ask whether that gap is statistically meaningful.

(We drop the "Other / unspecified" group here: you cannot draw a meaningful curve through 1–2 points.)

```
fig, ax = plt.subplots()
for g in ["Female", "Male"]:
    subset = users.loc[users["gender_label"] == g, "height_cm"].dropna()
    sns.kdeplot(subset, ax=ax, color=GENDER_COLORS[g], linewidth=2,
                label=f"{g} (n={len(subset)})")

ax.set_xlabel("Height (cm)")
ax.set_ylabel("Density")
ax.set_title("Density — height by gender", loc="left")
ax.legend(frameon=False)
plt.show()
```



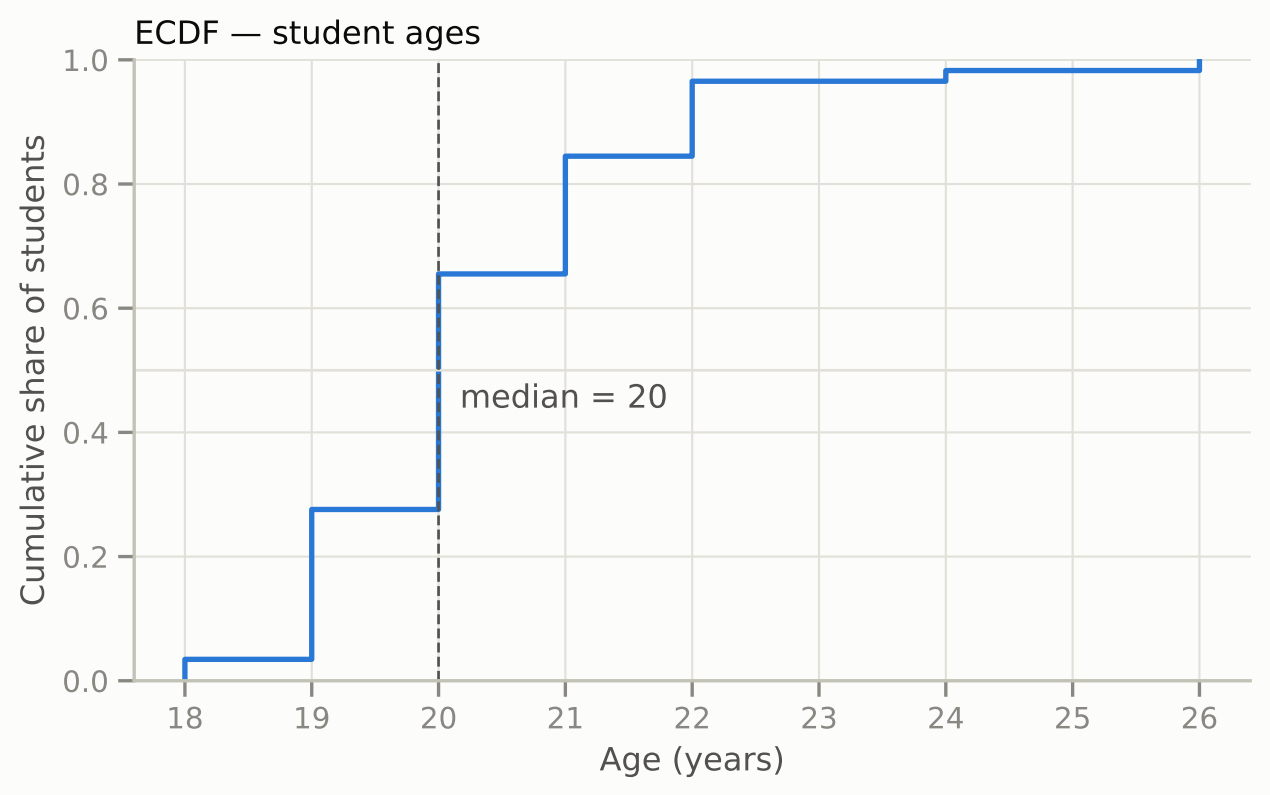
5.3 ECDF plot

The **empirical cumulative distribution function** answers: *what share of students is at or below a given value?* The curve always climbs from 0 to 1 as you move right.

How to read it: pick a point on the curve — say it passes through (20 years, 0.6). That means 60% of students are 20 or younger. The value where the curve crosses height 0.5 is the **median**. Steep sections = many students packed into that range; flat sections = few.

Unlike the histogram and KDE, the ECDF involves *no choices at all* (no bin width, no smoothing) — it shows the data exactly as it is.

```
fig, ax = plt.subplots()
sns.ecdfplot(users["age"], ax=ax, color=BLUE, linewidth=2)

median_age = users["age"].median()
ax.axhline(0.5, color=GRID, linewidth=1)
ax.axvline(median_age, color=INK2, linestyle="--", linewidth=1)
ax.annotate(f"median = {median_age:.0f}", xy=(median_age, 0.5),
            xytext=(8, -14), textcoords="offset points", color=INK2)

ax.set_xlabel("Age (years)")
ax.set_ylabel("Cumulative share of students")
ax.set_title("ECDF — student ages", loc="left")
plt.show()
```



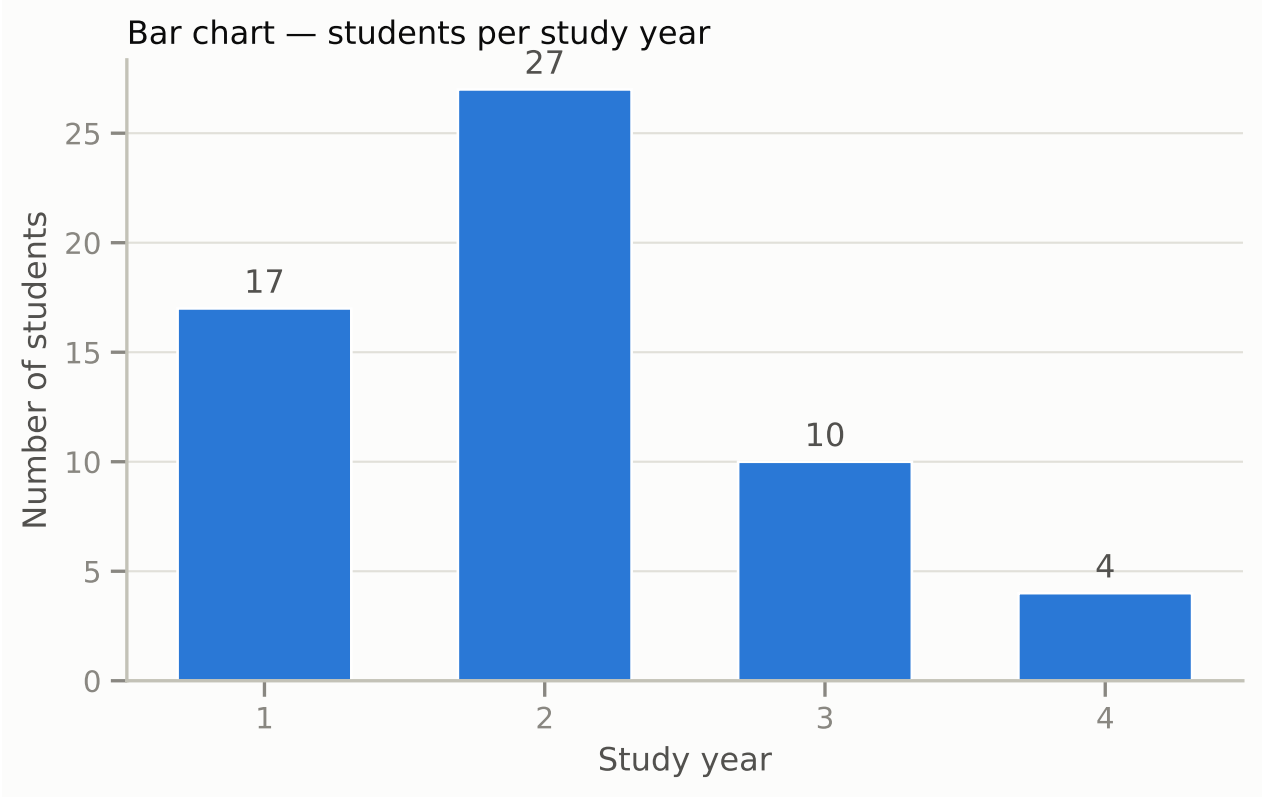
6. Comparing categories

6.1 Bar chart — ordered categories

Question: how big is each category? Bars compare counts (or averages) across groups. Three rules make bar charts honest and readable:

1. **Start the bars at zero.** The whole point of a bar is that its length is proportional to its value; a chopped-off axis makes small differences look huge.
2. **Label the bars directly** with their values, so nobody has to trace gridlines.
3. **Sort the bars by value — unless the categories have a natural order.** Study year is 1 → 2 → 3 → 4, a natural order, so we keep it.

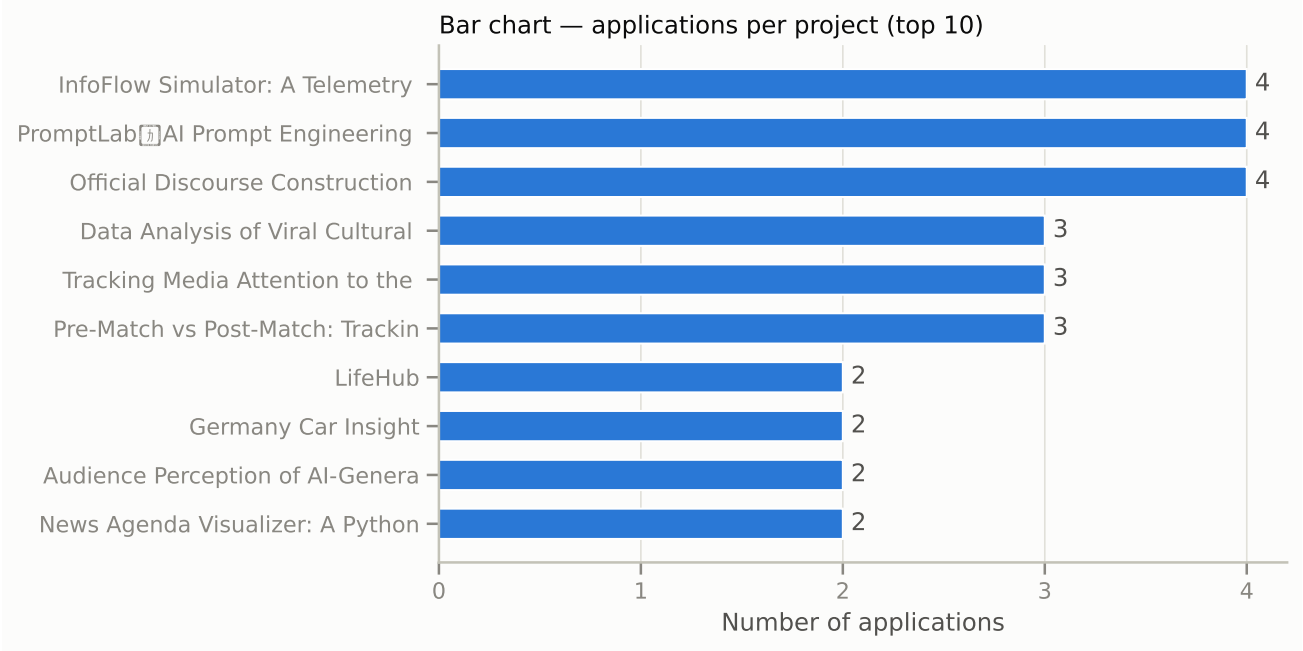
```
counts = users["study_year"].value_counts().sort_index()

fig, ax = plt.subplots()
bars = ax.bar(counts.index.astype(int).astype(str), counts.values,
              color=BLUE, width=0.62)
ax.bar_label(bars, padding=3, color=INK2)

ax.set_xlabel("Study year")
ax.set_ylabel("Number of students")
ax.set_title("Bar chart — students per study year", loc="left")
ax.grid(axis="x", visible=False)
plt.show()
```



6.2 Bar chart — unordered categories, sorted

Project titles have no natural order, so here we **do** sort by value. Two more practical tricks: horizontal bars keep long labels readable, and when there are many categories, showing the top 10 beats cramming in all 18.

```
top = (projects.assign(short_title=projects["title"].str.slice(0, 32))
               .sort_values("n_applications")
               .tail(10))                      # 10 most-applied-to projects

fig, ax = plt.subplots(figsize=(7.5, 4.6))
bars = ax.barh(top["short_title"], top["n_applications"], color=BLUE, height=0.62)
ax.bar_label(bars, padding=4, color=INK2)

ax.set_xlabel("Number of applications")
ax.set_title("Bar chart — applications per project (top 10)", loc="left")
ax.grid(axis="y", visible=False)
ax.xaxis.set_major_locator(MaxNLocator(integer=True))   # counts -> whole-number ticks
plt.show()
```



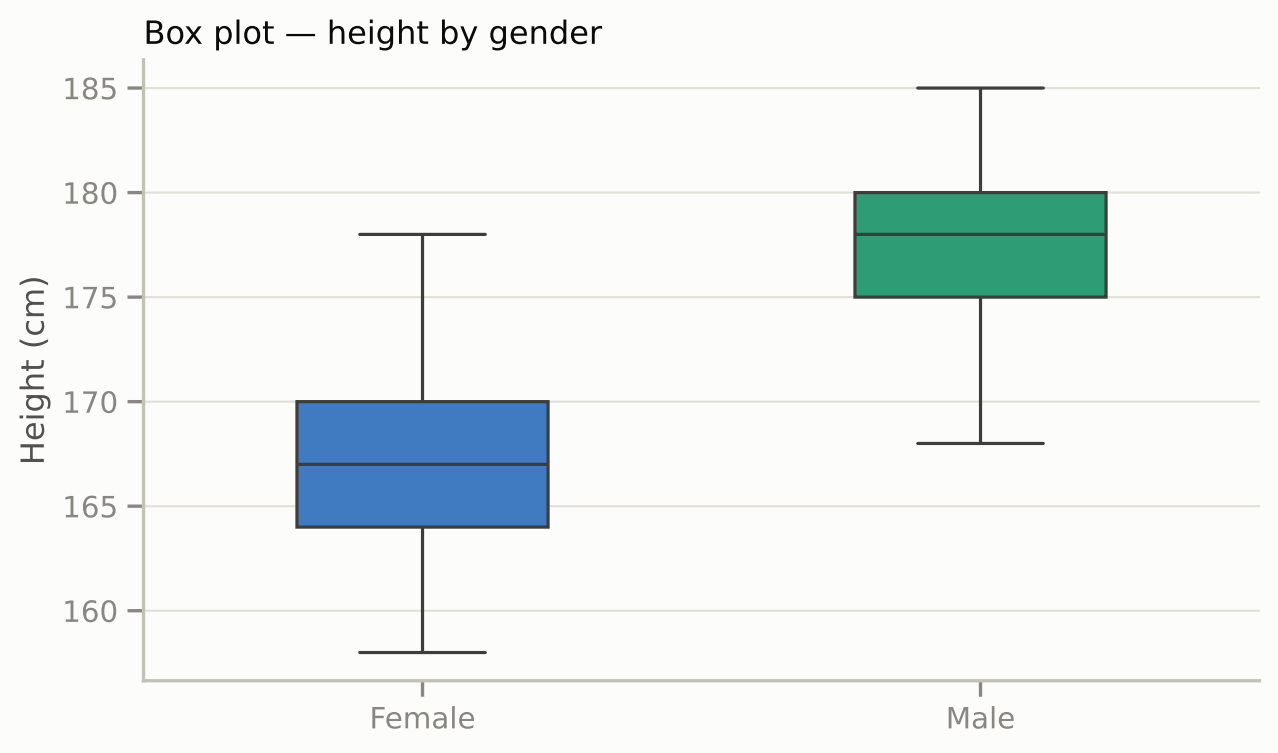
6.3 Box plot

*Question: how does the **whole distribution** of a number differ between groups?* A bar chart of group averages hides everything except the mean; the box plot shows five numbers at once. Reading one box from the inside out:

- the **line inside the box** = the median (middle value),
- the **box** = the middle half of the data, from the 25% to the 75% quartile (its height is called the *interquartile range*, IQR),
- the **whiskers** = the range of typical values,
- **dots beyond the whiskers** = outliers, individual unusually small/large values.

It is the picture version of the grouped table from section 3. Compare medians *and* how much the boxes overlap.

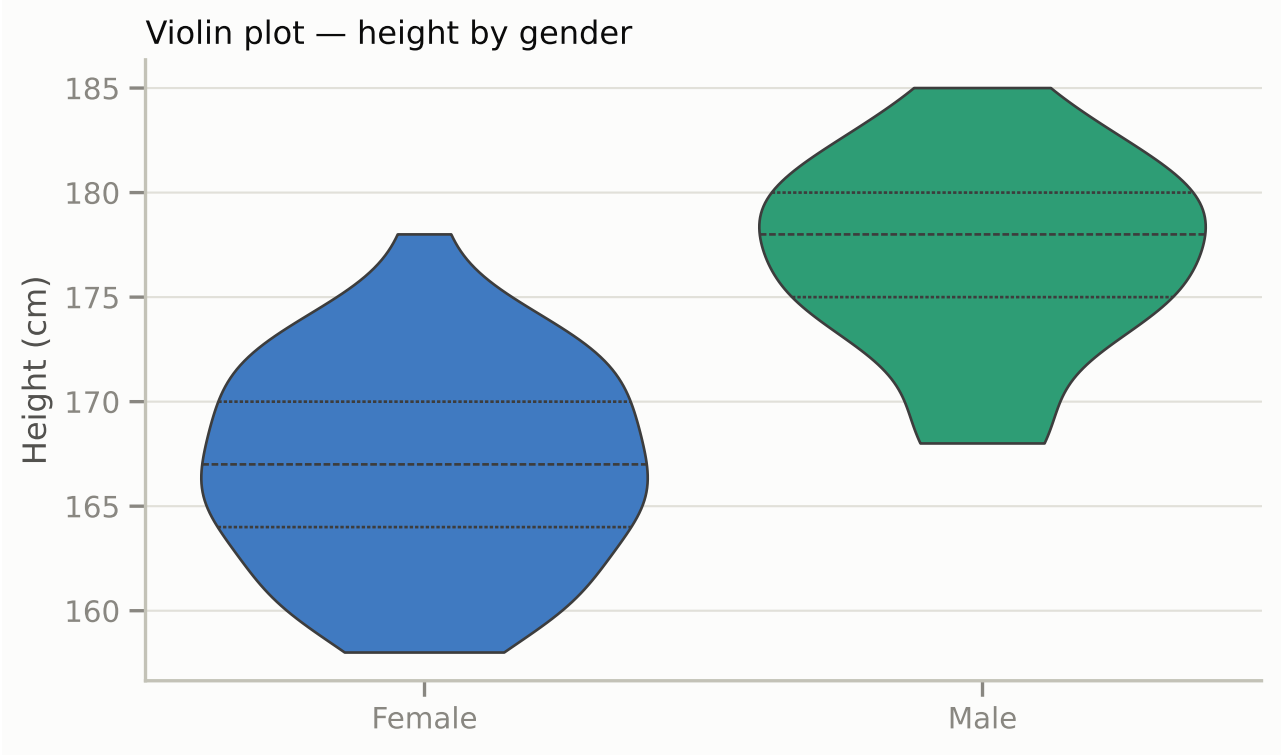
```
two_groups = users[users["gender_label"].isin(["Female", "Male"])]

fig, ax = plt.subplots()
sns.boxplot(data=two_groups, x="gender_label", y="height_cm",
            order=["Female", "Male"], hue="gender_label",
            palette=GENDER_COLORS, legend=False,
            width=0.45, linewidth=1.2, fliersize=3, ax=ax)

ax.set_xlabel("")
ax.set_ylabel("Height (cm)")
ax.set_title("Box plot —— height by gender", loc="left")
ax.grid(axis="x", visible=False)
plt.show()
```



6.4 Violin plot

The violin answers the same question as the box plot, but the **width of each shape shows how common values are at that height** — it is the KDE from section 5.2 turned sideways and mirrored. A box plot can hide a two-humped distribution; a violin cannot. The trade-off: violins need a decent number of points per group to be trustworthy (a violin of 5 points is mostly smoothing artifact).

```
fig, ax = plt.subplots()
sns.violinplot(data=two_groups, x="gender_label", y="height_cm",
               order=["Female", "Male"], hue="gender_label",
               palette=GENDER_COLORS, legend=False,
               inner="quart", linewidth=1, cut=0, ax=ax)

ax.set_xlabel("")
ax.set_ylabel("Height (cm)")
ax.set_title("Violin plot — height by gender", loc="left")
ax.grid(axis="x", visible=False)
plt.show()
```



7. Relationships and change over time

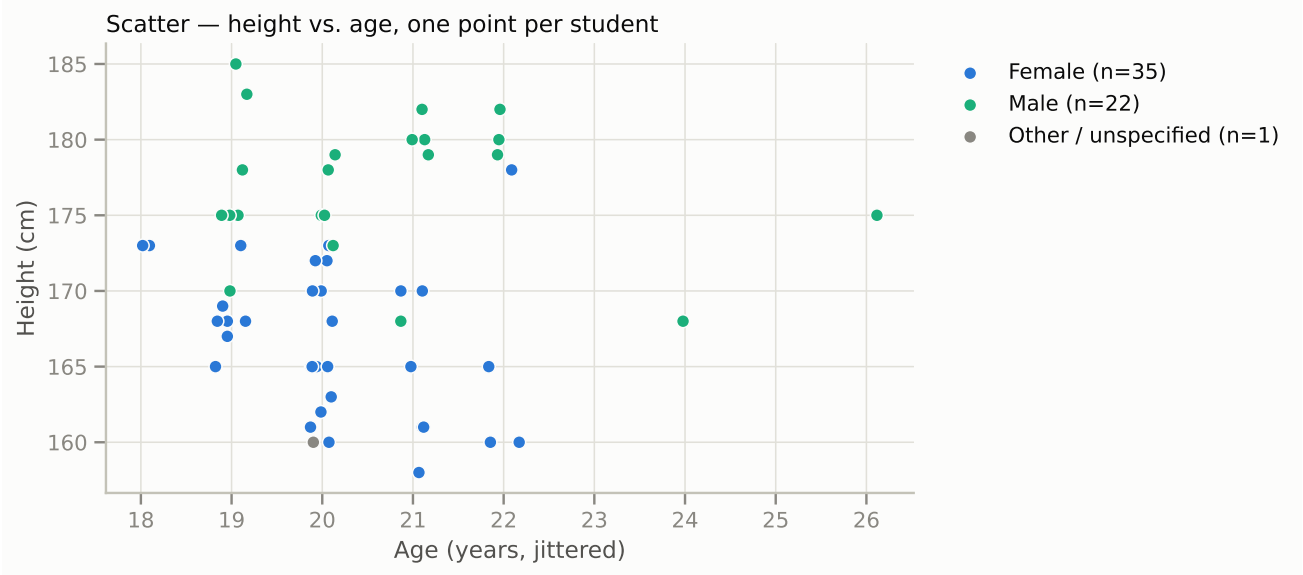
7.1 Scatter plot

Question: how do two numbers move together? Every dot is one **student** (check the grain!), placed at (their age, their height). If the cloud of dots drifts upward as you move right, the variables rise together; downward, they move oppositely; a shapeless cloud means no clear relationship.

Because ages are whole years, many students would land on exactly the same spot and hide each other — so we add a tiny random sideways nudge (**jitter**) to each dot, purely for visibility.

```
import numpy as np

rng = np.random.default_rng(42)
plot_df = users.dropna(subset=["age", "height_cm"]).copy()
plot_df["age_jitter"] = plot_df["age"] + rng.uniform(-0.18, 0.18, len(plot_df))

fig, ax = plt.subplots()
for g in GENDER_ORDER:
    sub = plot_df[plot_df["gender_label"] == g]
    ax.scatter(sub["age_jitter"], sub["height_cm"], s=45,
               color=GENDER_COLORS[g], label=f"{g} (n={len(sub)})",
               edgecolor=SURFACE, linewidth=0.8)

ax.set_xlabel("Age (years, jittered)")
ax.set_ylabel("Height (cm)")
```

```
ax.set_title("Scatter — height vs. age, one point per student", loc="left")
ax.legend(frameon=False, bbox_to_anchor=(1.02, 1), loc="upper left")
plt.show()
```

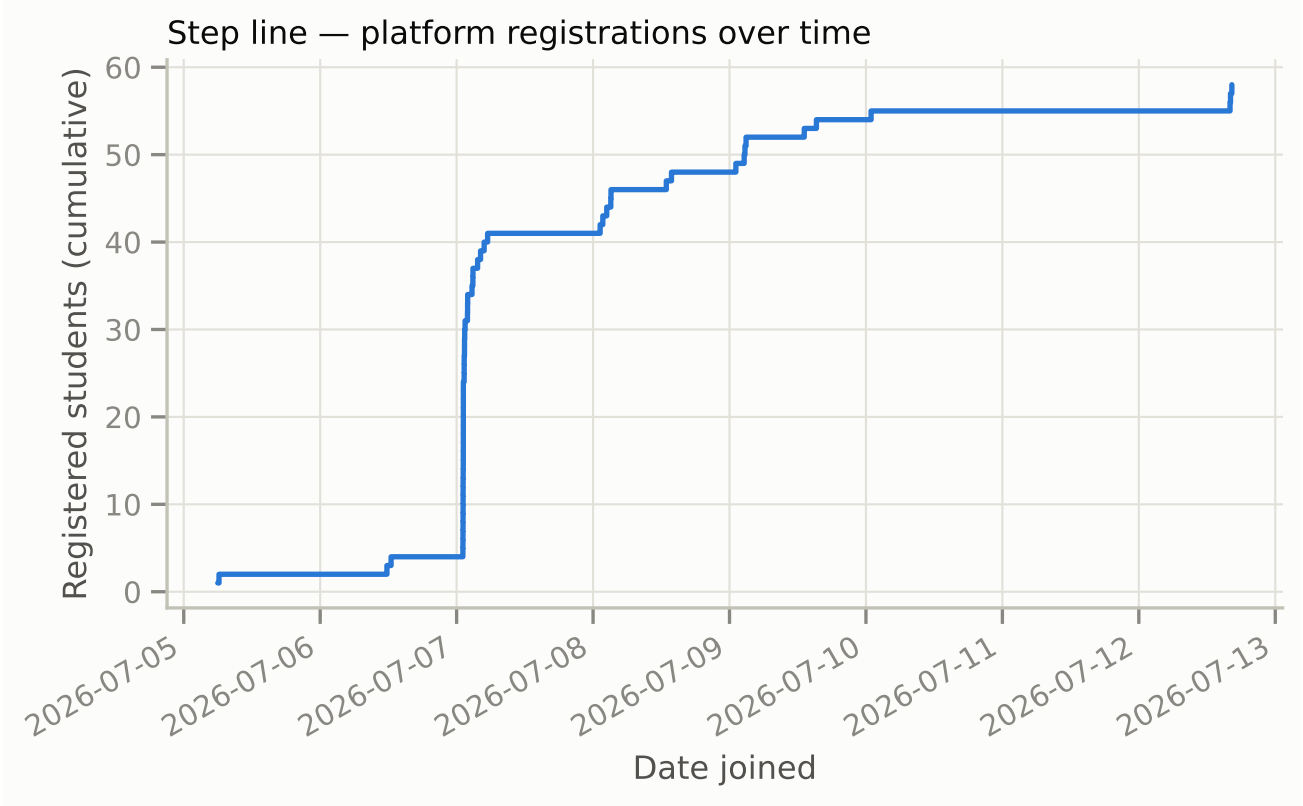


Within each color the cloud is fairly flat — age does not seem to matter for height — but the green cloud sits visibly above the blue one. Remember this: regression (section 9) will put numbers on exactly this picture.

7.2 Line / step plot over time

Question: how does a quantity change over time? Whenever rows carry a timestamp, you can plot growth. Here: registrations on the platform. We sort students by `date_joined` and count upward — the curve's **steep sections are the sign-up rushes** (you can probably guess which class deadlines caused them).

```
signups = users.sort_values("date_joined")

fig, ax = plt.subplots()
ax.step(signups["date_joined"], range(1, len(signups) + 1),
        where="post", color=BLUE, linewidth=2)

ax.set_xlabel("Date joined")
ax.set_ylabel("Registered students (cumulative)")
ax.set_title("Step line — platform registrations over time", loc="left")
fig.autofmt_xdate()
plt.show()
```



7.3 Correlation heatmap

Correlation (Pearson's r) compresses a whole scatter plot into one number between −1 and +1:

- r = +1: the dots lie exactly on a rising line; r = −1: exactly on a falling line;
- r = 0: no *linear* relationship;
- in between: the sign gives the direction, the size gives the strength (|r| above ~0.5 is usually called strong for this kind of data).

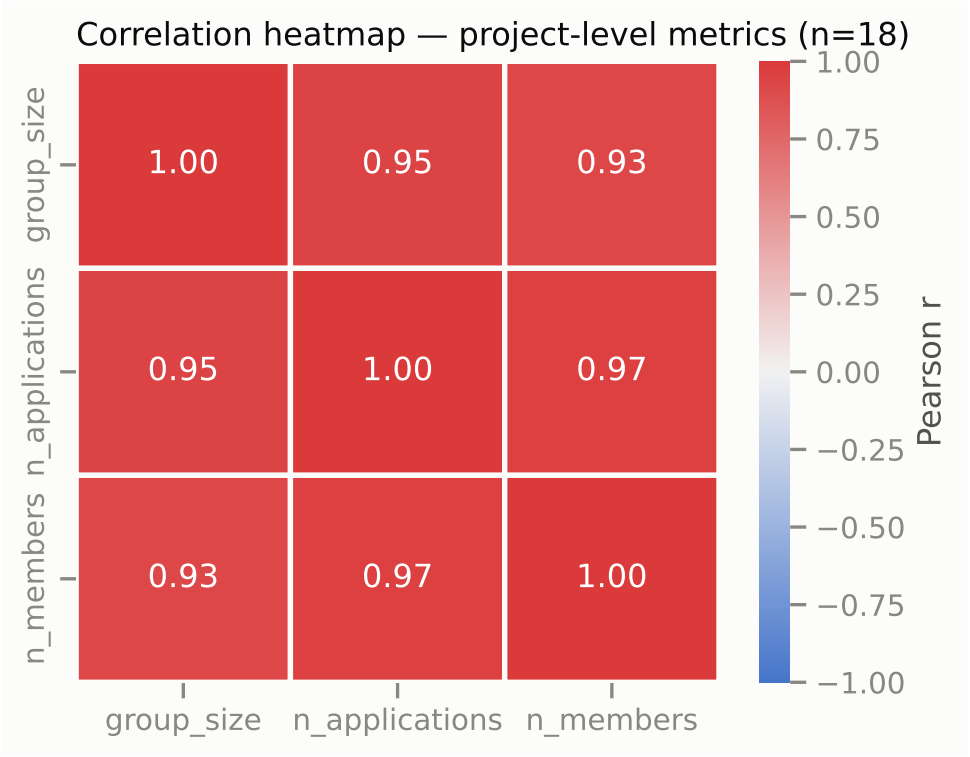
A heatmap shows r for **every pair of variables at once**. Here we switch to the *project* grain and ask: do projects that want more members (`group_size`) attract more applications, and end up with more members?

Design note: since r has a sign, the color scale must be **diverging** — one hue for negative, another for positive, meeting at neutral gray for 0. We also print the numbers in the cells, because color alone cannot show the difference between, say, 0.2 and 0.5.

Two cautions: with only 18 projects these numbers are rough descriptions, not proof; and **correlation is not causation** — "bigger group size goes with more applications" does not tell us *why*.

```
metrics = projects[["group_size", "n_applications", "n_members"]]
corr = metrics.corr()

fig, ax = plt.subplots(figsize=(5.4, 4.2))
sns.heatmap(corr, annot=True, fmt=".2f", center=0, vmin=-1, vmax=1,
            cmap=sns.diverging_palette(255, 12, as_cmap=True),
```

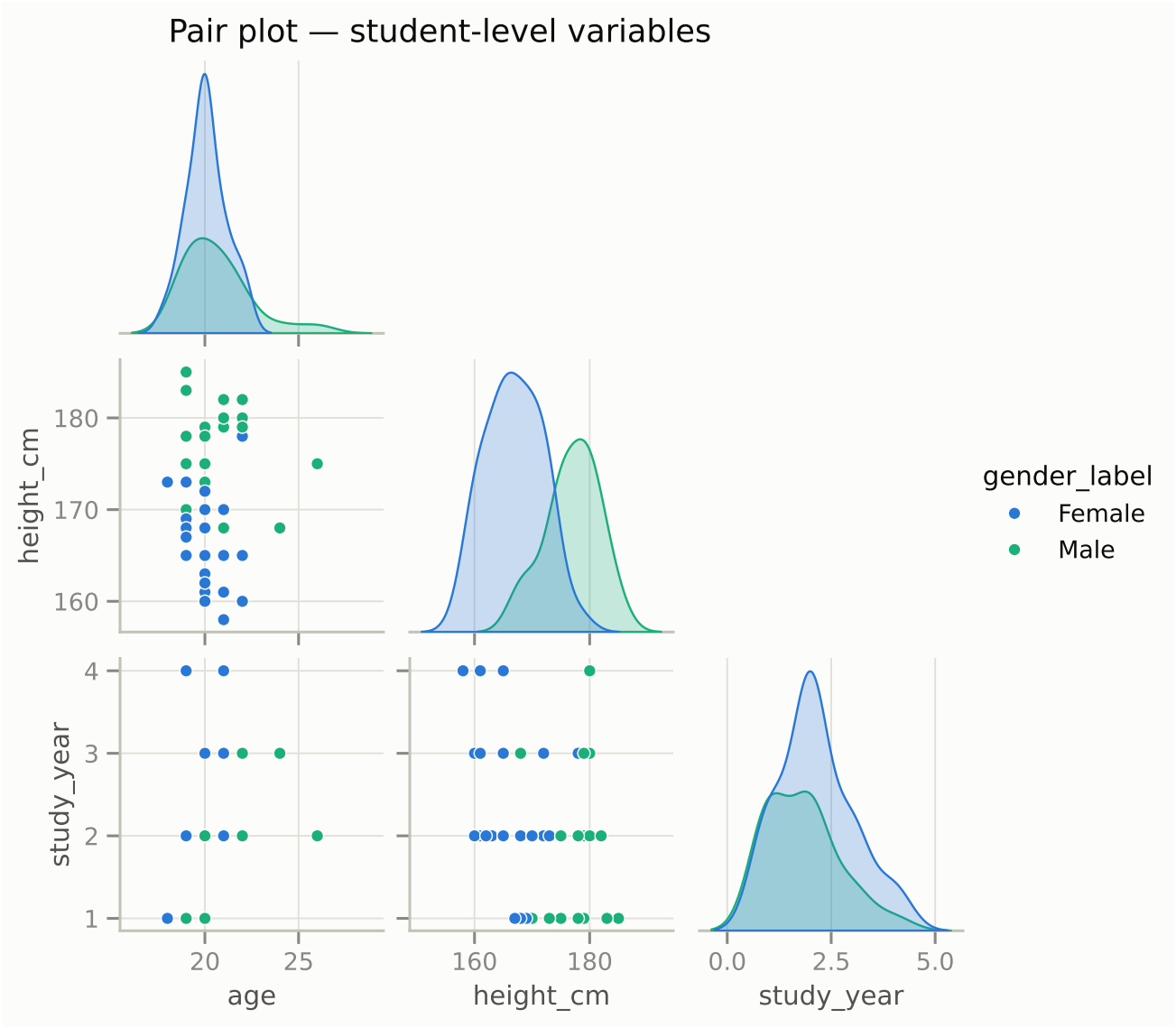
```
            linewidths=2, linecolor=SURFACE,
            cbar_kws={"label": "Pearson r"}, ax=ax)
ax.set_title("Correlation heatmap — project-level metrics (n=18)", loc="left")
plt.show()
```



7.4 Pair plot

The pair plot is a whole grid: every numeric variable scattered against every other, with each variable's distribution on the diagonal. Use it as a **first look** at a new dataset — a way to spot interesting pairs worth a proper plot of their own. It is too dense to be a final figure that communicates one message.

```
grid = sns.pairplot(two_groups[["age", "height_cm", "study_year", "gender_label"]].dropna(),
                    hue="gender_label", palette=GENDER_COLORS,
                    hue_order=["Female", "Male"],
                    height=2.1, corner=True,
                    plot_kws={"s": 32, "edgecolor": SURFACE, "linewidth": 0.6})
grid.figure.suptitle("Pair plot — student-level variables", x=0.38, y=1.02)
plt.show()
```



8. Hypothesis testing

Everything so far *described* our data. Statistical **inference** asks a sharper question: *is a pattern we see real, or could it just be luck?* Even if two groups were truly identical, random variation would make their sample averages differ a little — so how big does a difference have to be before we take it seriously?

The classic way to decide works like a courtroom:

1. **State the null hypothesis** H_0 — the "presumed innocent" position: *there is no difference / no association*.
2. **Collect evidence** — our data — and compute a test statistic.
3. **Compute the p-value** — the probability of seeing evidence *at least this strong* if H_0 were actually true. A tiny p-value means: "if there were really no difference, data like ours would be a huge coincidence."
4. **Compare with the significance level** α (the conventional threshold is 0.05). If $p < \alpha$, we **reject** H_0 — the pattern is called *statistically significant*. Otherwise we *fail to reject* H_0 — note

the careful wording: like a "not guilty" verdict, it means "not enough evidence", never "proven innocent".

One common misreading to avoid: the p-value is **not** "the probability that H_0 is true". It only measures how surprising our data would be *in a world where H_0 holds*.

Two honest caveats for this dataset: our **n is small** (weak evidence even for real effects), and the class is **not a random sample** of any wider population — so conclusions describe this class, not students in general.

8.0 Prepare the features

A few derived columns for the tests below: how many languages each student lists, whether they are active in any project, and an analysis table restricted to the two gender groups that are large enough to compare.

```
import numpy as np
from scipy import stats

# Number of languages listed ('Chinese, English' -> 2); split on common separators
users["n_languages"] = (users["languages"].fillna("")
                        .str.split(r"[,;/、和 &]+", regex=True)
                        .apply(lambda parts: sum(1 for p in parts if p.strip())))

# Active in any project: owns one, applied to one, or is a member of one
active_ids = (set(projects["owner_id"])
              | set(applications["applicant_id"])
              | set(memberships["member_id"]))
users["in_project"] = users["id"].isin(active_ids)

# Analysis frame: the two comparable gender groups, complete numeric fields
two = users[users["gender_label"].isin(["Female", "Male"])].dropna(
    subset=["height_cm", "age"]).copy()

print(f"analysis frame: {len(two)} students")
users["in_project"].value_counts()
```

```
analysis frame: 57 students


in_project
True     49
False     9
Name: count, dtype: int64
```

8.1 Two-sample t-test — do two group means differ?

Question: is mean height different for female and male students? The KDE in section 5.2 suggested a gap of about 10 cm — now we test it.

H_0: the two groups have the same mean height.

The tool is the **t-test**. Intuitively, it asks: *how large is the gap between the group averages, compared to how much heights naturally wobble within each group?* That ratio is the t-statistic; large |t| → small p-value. We use **Welch's version** (`equal_var=False`), which does not assume the two groups have equal spread — a safer default than the classic Student's t-test.

A good statistical report contains three numbers, not just one:

- the **p-value** — is the difference likely to be chance?
- the **confidence interval (CI)** of the difference — a range of plausible values for the true gap. "95% CI [7.5, 12.7]" reads: the data are compatible with a true gap between about 7.5 and 12.7 cm.
- an **effect size** (Cohen's d) — the gap measured in units of standard deviation, so it is comparable across studies. Rules of thumb: d ≈ 0.2 small, 0.5 medium, 0.8 large.

```
f = two.loc[two["gender_label"] == "Female", "height_cm"]
m = two.loc[two["gender_label"] == "Male", "height_cm"]
print(f"Female: n={len(f)}  mean={f.mean():.1f}  sd={f.std():.1f}")
print(f"Male:   n={len(m)}  mean={m.mean():.1f}  sd={m.std():.1f}")

res = stats.ttest_ind(m, f, equal_var=False)          # Welch's t-test
ci = res.confidence_interval()
print(f"\nWelch t = {res.statistic:.2f},  p = {res.pvalue:.2g}")
print(f"95% CI for mean difference (M - F): [{ci.low:.1f}, {ci.high:.1f}] cm")

# Effect size: Cohen's d with pooled standard deviation
sp = np.sqrt(((len(f) - 1) * f.var() + (len(m) - 1) * m.var()) / (len(f) + len(m) - 2))
print(f"Cohen's d = {(m.mean() - f.mean()) / sp:.2f}")
```

```
Female: n=35  mean=166.9  sd=4.9
Male:   n=22  mean=177.0  sd=4.6

Welch t = 7.92,  p = 3.5e-10
95% CI for mean difference (M - F): [7.5, 12.7] cm
Cohen's d = 2.13
```

Reading the result. The p-value is astronomically small (about 3 in 10 billion), so we reject H_0: this gap is not chance. The CI says the true difference is plausibly 7.5–12.7 cm, and d above 2 is a *very* large effect — unsurprising, since sex differences in height are one of the strongest effects in human biology. This is what a "textbook significant" result looks like.

The t-test assumes the groups are roughly bell-shaped (or reasonably large). When that is doubtful — strong skew, outliers, tiny groups — use the **Mann-Whitney U test**: it replaces the actual values by their *ranks* (1st smallest, 2nd smallest, …) and asks whether one group tends to hold the higher ranks. No bell shape required.

```
u = stats.mannwhitneyu(m, f, alternative="two-sided")
print(f"Mann-Whitney U = {u.statistic:.0f},  p = {u.pvalue:.2g}")
```

```
Mann-Whitney U = 720,  p = 3.9e-08
```

Both tests agree — a reassuring robustness check.

8.2 Chi-square test — are two categorical variables associated?

Question: is project participation related to gender? Both variables are categories (Female/Male, in-project True/False), so means and t-tests do not apply. Instead we build a **contingency table** — a grid of counts for every combination — and ask:

H_0**: the two variables are independent** (participation rates are the same for every gender).

The **chi-square (χ^2) test** compares the **observed** counts with the counts we would **expect if H_0 were true** (computed from the row and column totals). Big gaps between observed and expected → large χ^2 → small p-value.

Fine print: the chi-square approximation is only trustworthy when every *expected* count is at least about 5. Small tables like ours often violate that — the fix is **Fisher's exact test**, which computes the exact probability for a 2×2 table with no approximation.

```
ct = pd.crosstab(two["gender_label"], two["in_project"])
print("Observed counts:")
print(ct)

chi2, p, dof, expected = stats.chi2_contingency(ct)
print(f"\nchi-square = {chi2:.2f},  dof = {dof},  p = {p:.2g}")
print("\nExpected counts under independence:")
print(pd.DataFrame(expected, index=ct.index, columns=ct.columns).round(1))

# Small 2x2 table -> Fisher's exact test is the more trustworthy answer
odds, p_fisher = stats.fisher_exact(ct)
print(f"\nFisher's exact test: odds ratio = {odds:.2f},  p = {p_fisher:.2g}")
```

```
Observed counts:
in_project    False  True
gender_label
Female            6     29
Male              2     20

chi-square = 0.21,  dof = 1,  p = 0.65

Expected counts under independence:
in_project    False  True
gender_label
Female          4.9   30.1
```

```
Male            3.1   18.9

Fisher's exact test: odds ratio = 2.07,  p = 0.47
```

Reading the result. The p-value is large (far above 0.05), so we fail to reject H_0: the observed table is entirely compatible with "participation does not depend on gender". Also notice one expected count is below 5 — exactly the situation where you should quote Fisher's exact test (which agrees: no association). A non-significant result is not a failure; "we found no evidence of a difference" is a perfectly good scientific answer.

9. Regression

Hypothesis tests answer *"is there a difference?"*. **Regression** answers the richer question: *"**how** does an outcome depend on one or more predictors — and by how much?"*

9.1 Simple linear regression

Linear regression fits the best straight line through a scatter cloud:

$$\text{height} = a + b \cdot \text{age}$$

- a (the **intercept**) is the predicted height when age = 0 — often not meaningful by itself, just where the line starts;
- b (the **slope/coefficient**) is the interesting one: *predicted change in height for each extra year of age*.

"Best" line means: the line that makes the total (squared) prediction errors as small as possible — *ordinary least squares* (OLS).

The `statsmodels` formula syntax reads like the model itself: outcome on the left of ~, predictors on the right.

```python
import statsmodels.api as sm
import statsmodels.formula.api as smf

m1 = smf.ols("height_cm ~ age", data=two).fit()
print(m1.summary())
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:              height_cm   R-squared:                       0.000
Model:                            OLS   Adj. R-squared:                 -0.018
Method:                 Least Squares   F-statistic:                   0.02107
Date:                Tue, 14 Jul 2026   Prob (F-statistic):              0.885
Time:                        15:59:21   Log-Likelihood:                -190.04
No. Observations:                  57   AIC:                             384.1
Df Residuals:                      55   BIC:                             388.2
```

```
Df Model:                           1
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
Intercept    172.7493     13.533     12.765      0.000     145.629     199.870
age           -0.0966      0.665     -0.145      0.885      -1.430       1.236
==============================================================================
Omnibus:                        4.592   Durbin-Watson:                   1.774
Prob(Omnibus):                  0.101   Jarque-Bera (JB):                2.082
Skew:                           0.101   Prob(JB):                        0.353
Kurtosis:                       2.086   Cond. No.                         302.
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
```

The summary prints a lot; for now, three numbers matter:

- **coef** in the `age` row — the slope b: predicted cm of height per extra year of age.
- **P>|t|** in the `age` row — a p-value for H_0: *the true slope is 0* (i.e. age tells us nothing). Here it is large → age is not a useful predictor. That matches the flat scatter in section 7.1, and makes sense: our students are all adults, who have stopped growing.
- **R-squared** (R^2) — the share of the height variation the model explains, from 0 to 1. Near 0 here: knowing a student's age barely improves your guess of their height.

A weak first predictor is not the end of the story — regression becomes powerful when we add the *right* one.

9.2 Multiple regression — adding a categorical predictor

The scatter plot showed that *gender*, not age, separates the heights. Categorical predictors enter a regression as **dummy variables** — columns of 0/1. `C(gender_label)` does this automatically: it picks a baseline category (Female, alphabetically) and adds a 0/1 column "is this student Male?".

$$\text{height} = a + b_1 \cdot \text{age} + b_2 \cdot \text{Male}$$

b_2 then has a very concrete meaning: *comparing two students of the same age, the Male one is predicted to be b_2 cm taller.* This "holding other variables constant" reading is what makes multiple regression so useful.

```
m2 = smf.ols("height_cm ~ age + C(gender_label)", data=two).fit()
print(m2.summary())
```

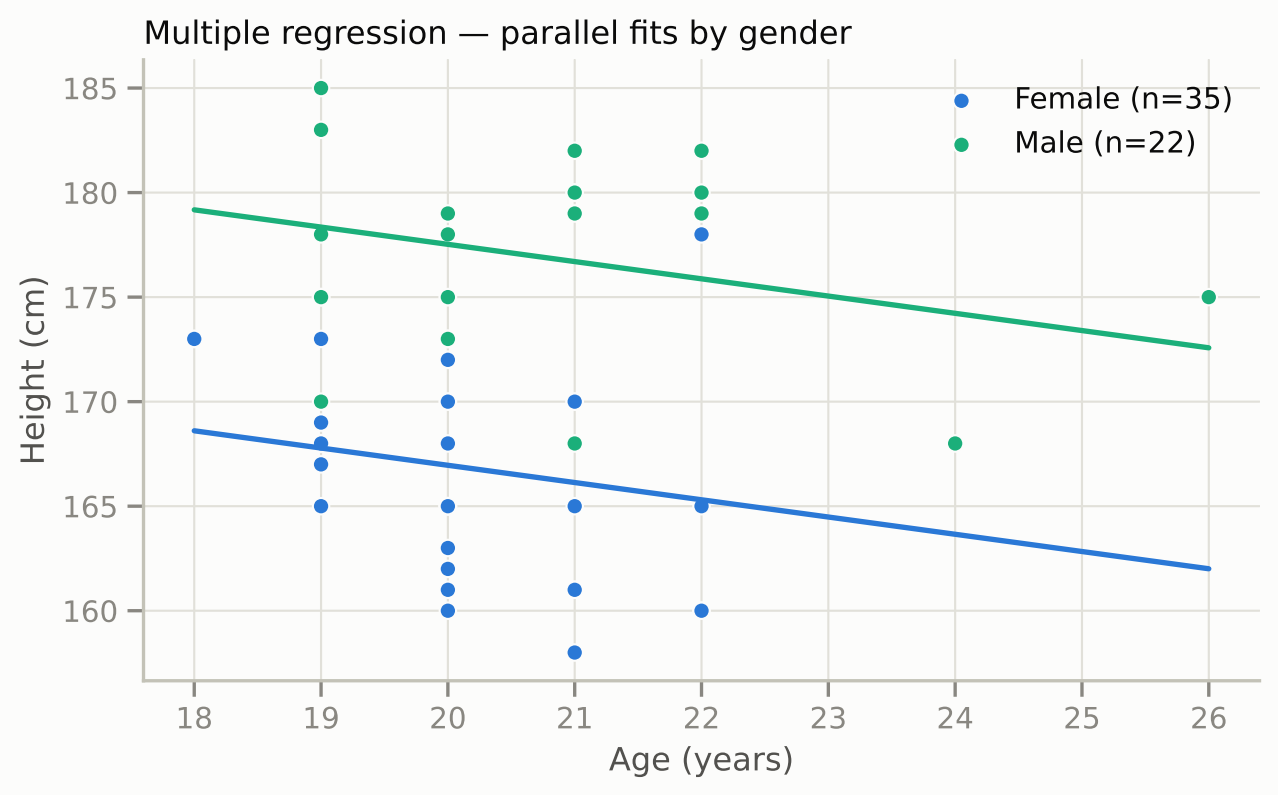
```
                            OLS Regression Results
==============================================================================
Dep. Variable:              height_cm   R-squared:                       0.553
```

```
Model:                            OLS   Adj. R-squared:                  0.536
Method:                 Least Squares   F-statistic:                     33.38
Date:                Tue, 14 Jul 2026   Prob (F-statistic):           3.65e-10
Time:                        15:59:21   Log-Likelihood:                -167.12
No. Observations:                  57   AIC:                             340.2
Df Residuals:                      54   BIC:                             346.4
Df Model:                           2
Covariance Type:            nonrobust
================================================================================================
                                   coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------------------------
Intercept                    183.4553      9.228     19.880      0.000     164.954     201.956
C(gender_label)[T.Male]       10.5685      1.294      8.168      0.000       7.974      13.163
age                           -0.8249      0.458     -1.802      0.077      -1.743       0.093
==============================================================================
Omnibus:                        0.384   Durbin-Watson:                   1.871
Prob(Omnibus):                  0.825   Jarque-Bera (JB):                0.554
Skew:                           0.110   Prob(JB):                        0.758
Kurtosis:                       2.570   Cond. No.                         305.
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
```

Reading the result. The `C(gender_label)[T.Male]` coefficient is close to the ~10 cm gap we measured with the t-test (reassuring — same data, same story, different tool), and its p-value is tiny. Age remains non-significant. And compare R^2 with model 1: from almost nothing to roughly half the variation explained — adding the *right* predictor did what adding *more* of a wrong one never could.

Geometrically, this model is two **parallel lines** — same age slope, different heights:

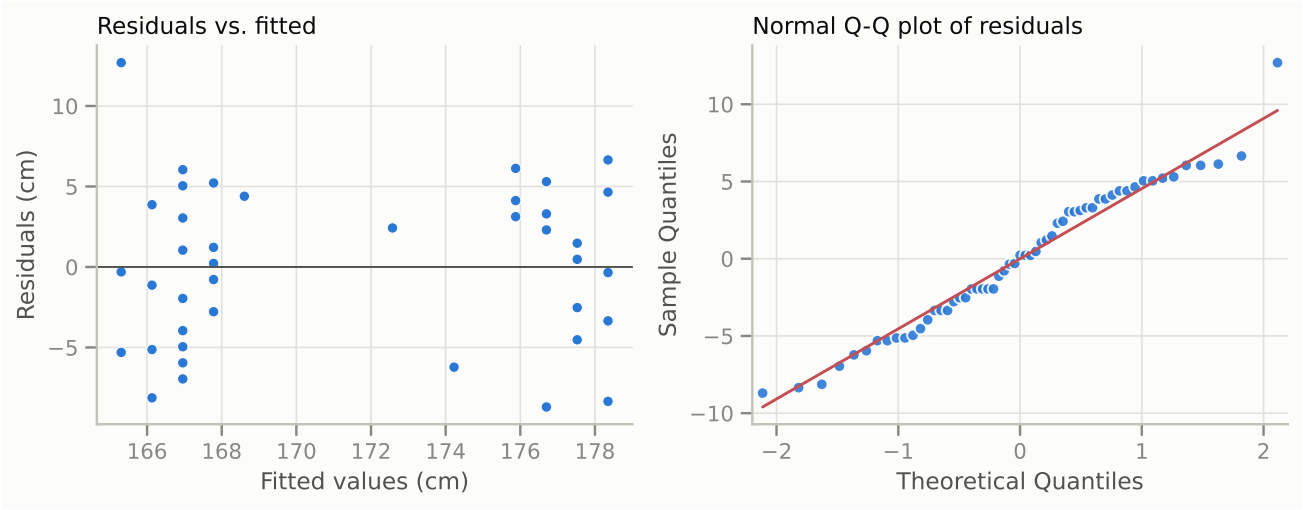
```
fig, ax = plt.subplots()
ages = np.linspace(two["age"].min(), two["age"].max(), 50)
for g in ["Female", "Male"]:
    sub = two[two["gender_label"] == g]
    ax.scatter(sub["age"], sub["height_cm"], s=38, color=GENDER_COLORS[g],
               edgecolor=SURFACE, linewidth=0.8, label=f"{g} (n={len(sub)})")
    pred = m2.predict(pd.DataFrame({"age": ages, "gender_label": g}))
    ax.plot(ages, pred, color=GENDER_COLORS[g], linewidth=2)

ax.set_xlabel("Age (years)")
ax.set_ylabel("Height (cm)")
ax.set_title("Multiple regression — parallel fits by gender", loc="left")
ax.legend(frameon=False)
plt.show()
```



Always check the residuals. A **residual** is one student's prediction error: actual height minus predicted height. OLS is trustworthy when the residuals look like harmless random noise. Two standard pictures:

- **Residuals vs. fitted values** — should be a shapeless cloud around 0. A funnel (spread growing with fitted value) or a curve means the model is missing something.
- **Normal Q-Q plot** — compares your residuals' quantiles with a perfect bell curve's; points hugging the straight line = roughly normal errors, which is what the p-values above assume.

```
fig, axes = plt.subplots(1, 2, figsize=(9.5, 3.8))

axes[0].scatter(m2.fittedvalues, m2.resid, s=30, color=BLUE,
                edgecolor=SURFACE, linewidth=0.6)
axes[0].axhline(0, color=INK2, linewidth=1)
axes[0].set_xlabel("Fitted values (cm)")
axes[0].set_ylabel("Residuals (cm)")
axes[0].set_title("Residuals vs. fitted", loc="left")

sm.qqplot(m2.resid, line="s", ax=axes[1], markerfacecolor=BLUE,
          markeredgecolor=SURFACE, alpha=0.9)
axes[1].set_title("Normal Q-Q plot of residuals", loc="left")

plt.tight_layout()
plt.show()
```



Here both plots look fine: no funnel, no curve, points near the Q-Q line — the model's assumptions are reasonable.

9.3 Logistic regression — when the outcome is yes/no

Linear regression needs a numeric outcome. But many interesting outcomes are **binary**: passed/failed, joined/didn't, clicked/didn't. Fitting a straight line to 0/1 data quickly predicts nonsense like "probability 1.4" — so instead, **logistic regression** fits an S-shaped curve that always stays between 0 and 1, and models the *probability* of the "1" outcome.

Our demo: *model the probability that a student belongs to the male group, given their height.* (This is a statistics exercise, not a claim about individuals — the two height distributions overlap heavily, which is exactly why a *probability* model is the right tool.)

One new concept before the output: **odds**. If an event has probability 0.75, its odds are 0.75/0.25 = 3 ("three to one"). Logistic regression coefficients live on the (log-)odds scale, which makes them awkward to read raw — but e^{coef} has a clean meaning: the **odds ratio**, i.e. *each extra unit of the predictor multiplies the odds by this factor*.

```
two["is_male"] = (two["gender_label"] == "Male").astype(int)

logit = smf.logit("is_male ~ height_cm", data=two).fit()
print(logit.summary())

or_per_cm = np.exp(logit.params["height_cm"])
print(f"\nOdds ratio: each extra cm multiplies the odds of 'male' by {or_per_cm:.2f}")
```

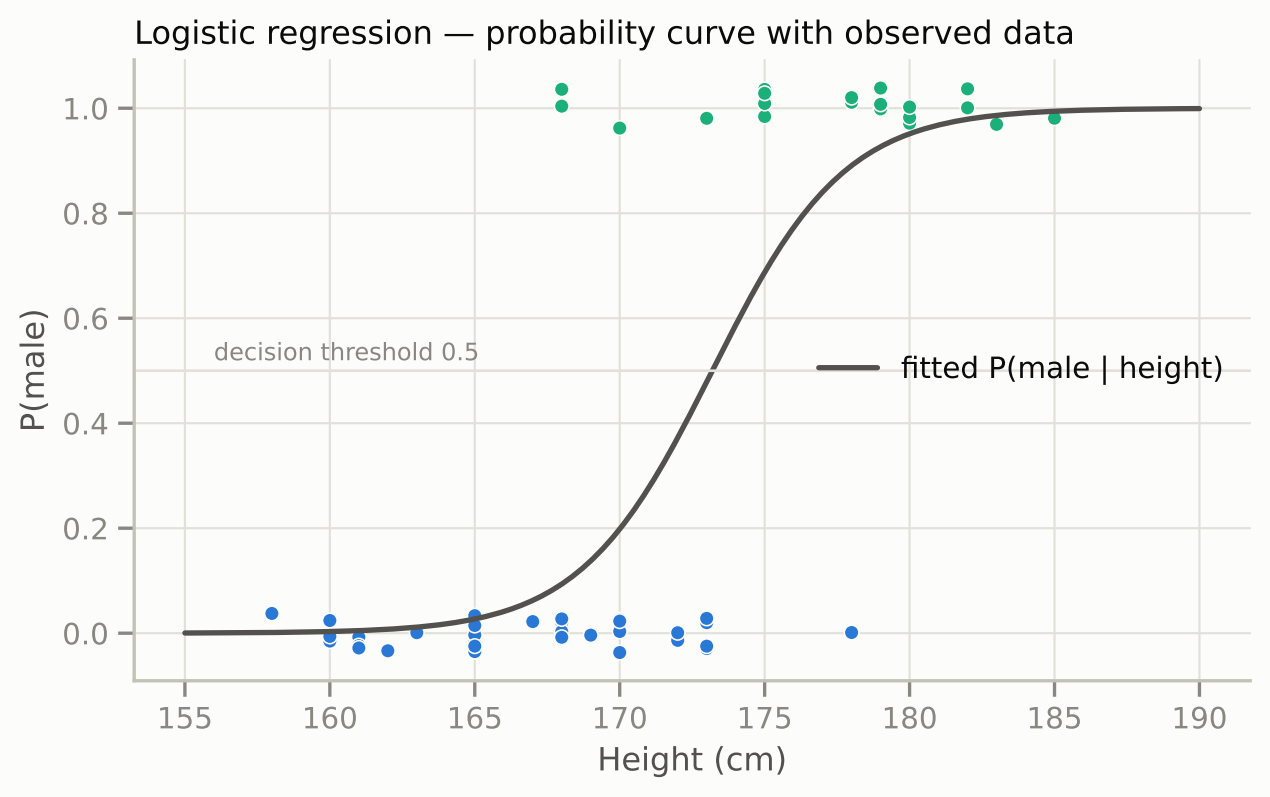
```
Optimization terminated successfully.
         Current function value: 0.318738
         Iterations 8
                           Logit Regression Results
==============================================================================
Dep. Variable:                is_male   No. Observations:                   57
Model:                          Logit   Df Residuals:                       55
```

```
Method:                           MLE   Df Model:                            1
Date:                Tue, 14 Jul 2026   Pseudo R-squ.:                  0.5221
Time:                        15:59:21   Log-Likelihood:                -18.168
converged:                       True   LL-Null:                       -38.014
Covariance Type:            nonrobust   LLR p-value:                 2.974e-10
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
Intercept    -75.7049     19.897     -3.805      0.000    -114.703     -36.707
height_cm      0.4371      0.115      3.795      0.000       0.211       0.663
==============================================================================

Odds ratio: each extra cm multiplies the odds of 'male' by 1.55
```

Reading the result. The height coefficient is positive and significant: taller → higher probability of being in the male group. The odds ratio says each additional centimeter multiplies those odds by roughly 1.6–1.7. (`Pseudo R-squ.` plays the role of R^2; around 0.5 is high for a logistic model.)

The fitted curve makes it visual — each dot is a student at their height, drawn at 0 (female) or 1 (male), and the curve is the model's estimated probability:

```python
heights = np.linspace(155, 190, 200)
p_hat = logit.predict(pd.DataFrame({"height_cm": heights}))

rng = np.random.default_rng(1)
jitter = rng.uniform(-0.04, 0.04, len(two))

fig, ax = plt.subplots()
ax.plot(heights, p_hat, color=INK2, linewidth=2, label="fitted P(male | height)")
ax.scatter(two["height_cm"], two["is_male"] + jitter, s=30,
           c=two["gender_label"].map(GENDER_COLORS),
           edgecolor=SURFACE, linewidth=0.6)
ax.axhline(0.5, color=GRID, linewidth=1)
ax.annotate("decision threshold 0.5", xy=(156, 0.52), color=MUTED, fontsize=9)

ax.set_xlabel("Height (cm)")
ax.set_ylabel("P(male)")
ax.set_title("Logistic regression — probability curve with observed data", loc="left")
ax.legend(frameon=False, loc="center right")
plt.show()
```



From probabilities to predictions. To use the model as a classifier, predict "male" whenever the fitted probability exceeds 0.5, then count how often that matches reality — a **confusion matrix**. One trap to avoid: always compare accuracy with the **majority-class baseline** — the accuracy of mindlessly guessing the larger group every time. A model is only good if it clearly beats that.

```
pred = (logit.predict(two) >= 0.5).astype(int)

print(pd.crosstab(two["is_male"], pred,
                  rownames=["actual"], colnames=["predicted"]))
baseline = max(two["is_male"].mean(), 1 - two["is_male"].mean())
print(f"\naccuracy = {(pred == two['is_male']).mean():.0%}"
      f"   (majority-class baseline = {baseline:.0%})")
```

```
predicted   0   1
actual
0          34   1
1           4  18

accuracy = 91%   (majority-class baseline = 61%)
```

The diagonal of the table (top-left and bottom-right) counts correct predictions; the off-diagonal cells are the mistakes. Around 90% accuracy against a ~60% baseline: height alone is a genuinely informative predictor here.

9.4 Exercises

Try these yourself — every ingredient is already in this notebook:

1. **ANOVA** (`stats.f_oneway`) — the t-test's big sibling for 3+ groups: does mean height differ across study years 1–4?
2. **Chi-square**: is `study_year` associated with `gender_label`? Check the expected counts first — is the test even valid here?
3. **Linear regression**: `n_languages ~ study_year` — do senior students list more languages?
4. **Logistic regression**: `in_project ~ study_year + C(gender_label) + n_languages` — does anything predict who joins a project? (Expect wide confidence intervals: n is small and most students participate.)

10. Cheat sheet

Which plot for which question?

You want to show…	Use	Section
The distribution of one numeric variable	Histogram / KDE / ECDF	5.1–5.3
Sizes of categories	Bar chart (zero-based, labeled; sort unordered categories)	6.1–6.2
A numeric distribution *per category*	Box plot (summary) / violin (shape)	6.3–6.4
The relationship between two numeric variables	Scatter plot	7.1
Change over time	Line / step plot	7.2
All pairwise linear relationships	Correlation heatmap / pair plot	7.3–7.4

Which test / model for which question?

You want to know…	Method	Section
Do two group means differ?	Welch t-test (Mann-Whitney if non-normal)	8.1
Are two categorical variables associated?	Chi-square (Fisher's exact for small 2×2)	8.2
How does a numeric outcome depend on predictors?	Linear regression (OLS) + residual checks	9.1–9.2
How does a binary outcome depend on predictors?	Logistic regression, odds ratios	9.3

Habits worth keeping:

- Know the **grain** of each table (per student vs. per project) before plotting.

- Look at the **numbers first** (`describe`, `value_counts`), plots second, tests last.
- Keep **one fixed color per category** across all figures.
- **Free-text columns are not categories** until cleaned (section 3.1).
- Drop or merge groups too small to summarize — and say so.
- Report **effect sizes and confidence intervals** alongside p-values — "significant" is not the same as "large".
- Check regression **residuals** before trusting coefficients.
- **Correlation is not causation**, and with small n, results describe *this class* — they don't estimate a population.

Finally, release the database connections held by this notebook:

```
engine.dispose()
```